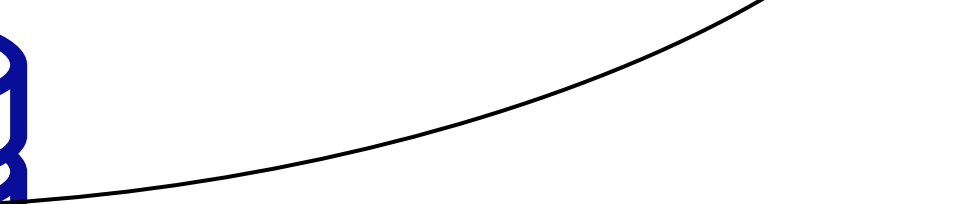
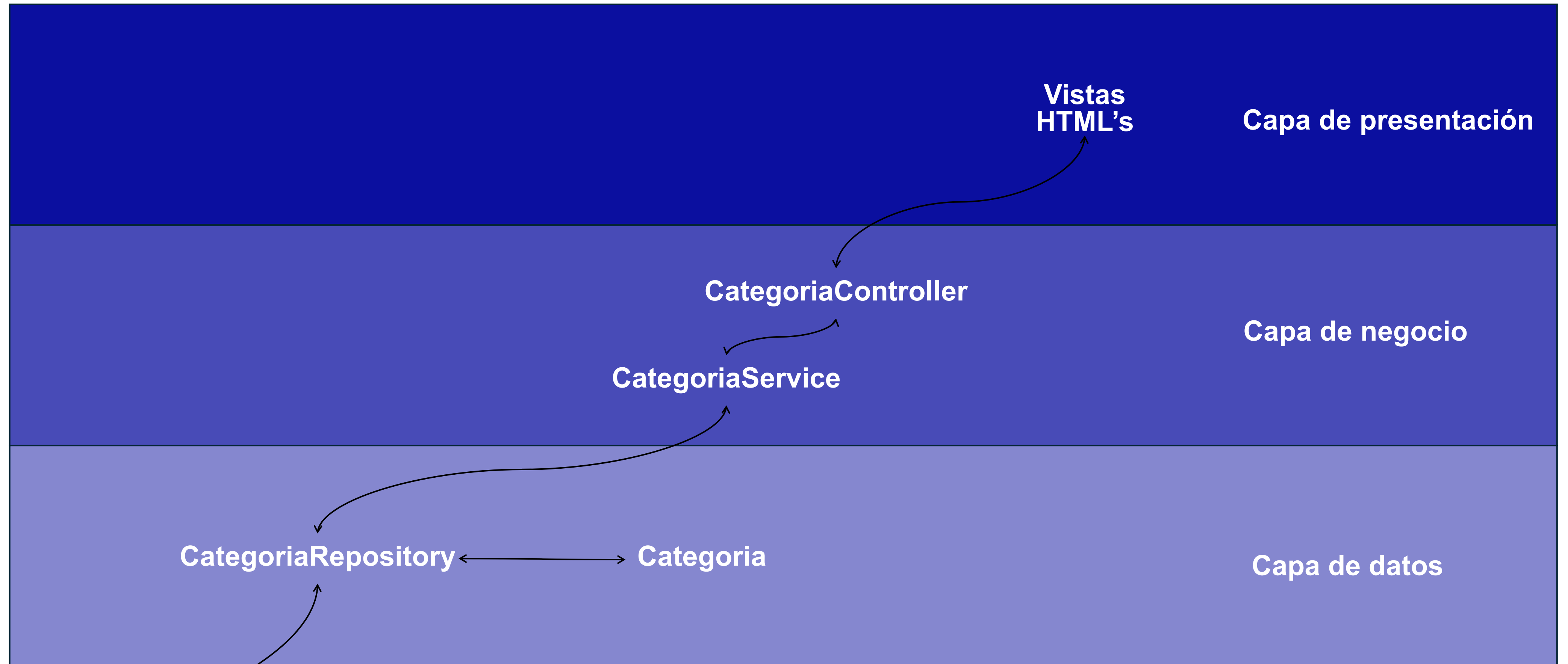


Web y Patrones Semana 3



Patrón de diseño MVC

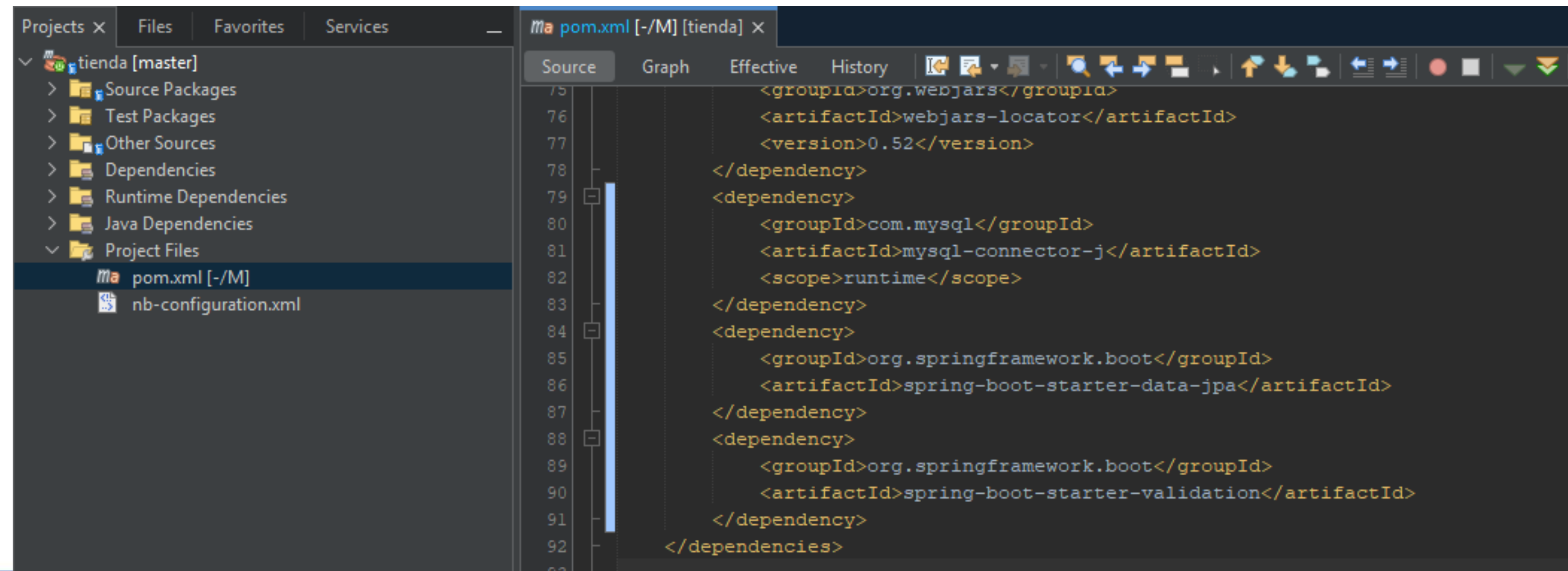


Depentencias

pom.xml

Se incorporan dos nuevas dependencias para poder utilizar MySQL e Hibernate por medio de JPA.

Se incorporan las dependencias contenidas entre las líneas 79 a 91, según la imagen adjunta.

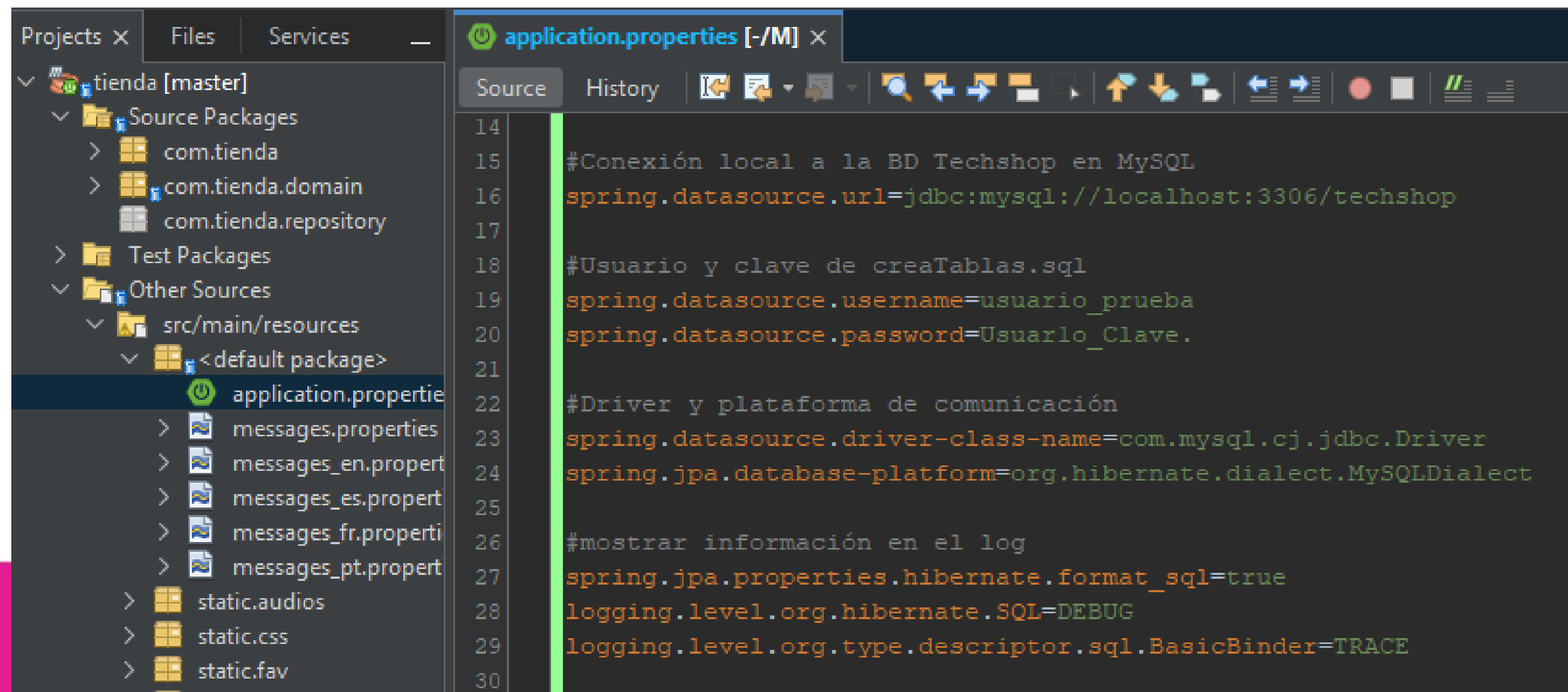


The screenshot shows an IDE window with the file `ma pom.xml [-/M] [tienda] x` open. The left sidebar shows the project structure with `tienda [master]` expanded, showing `Source Packages`, `Test Packages`, `Other Sources`, `Dependencies`, `Runtime Dependencies`, `Java Dependencies`, and `Project Files`. The `Project Files` section shows `ma pom.xml [-/M]` and `nb-configuration.xml`. The main editor shows the XML content of `ma pom.xml` with the following dependencies added between lines 79 and 91:

```
75 <groupId>org.webjars</groupId>
76 <artifactId>webjars-locator</artifactId>
77 <version>0.52</version>
78 </dependency>
79 <dependency>
80 <groupId>com.mysql</groupId>
81 <artifactId>mysql-connector-j</artifactId>
82 <scope>runtime</scope>
83 </dependency>
84 <dependency>
85 <groupId>org.springframework.boot</groupId>
86 <artifactId>spring-boot-starter-data-jpa</artifactId>
87 </dependency>
88 <dependency>
89 <groupId>org.springframework.boot</groupId>
90 <artifactId>spring-boot-starter-validation</artifactId>
91 </dependency>
92 </dependencies>
```

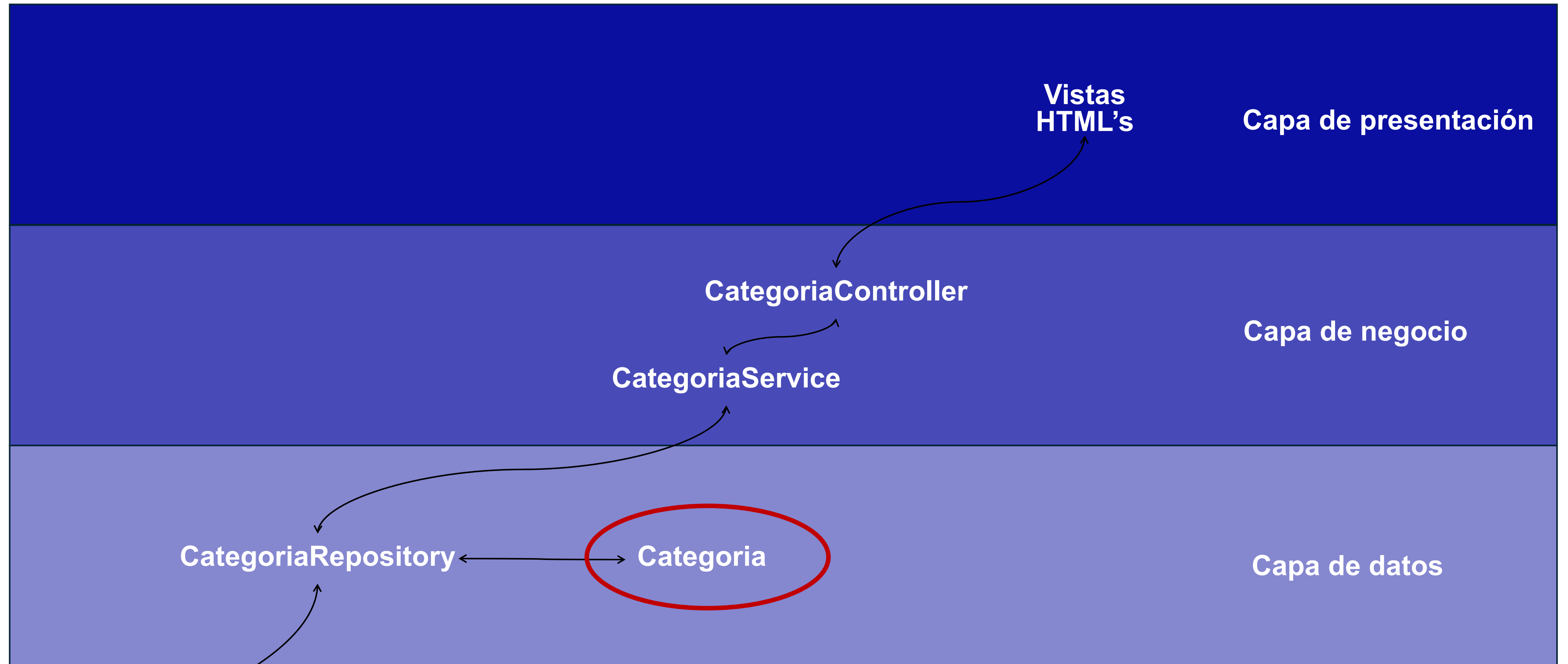
application.properties

Se agregan instrucciones. En el archivo **application.properties** se incorpora lo necesario para realizar la conexión a MySQL y para mostrar información en el log.



```
14
15 #Conexión local a la BD Techshop en MySQL
16 spring.datasource.url=jdbc:mysql://localhost:3306/techshop
17
18 #Usuario y clave de creaTablas.sql
19 spring.datasource.username=usuario_prueba
20 spring.datasource.password=Usuario_Clave.
21
22 #Driver y plataforma de comunicación
23 spring.datasource.driver-class-name=com.mysql.cj.jdbc.Driver
24 spring.jpa.database-platform=org.hibernate.dialect.MySQLDialect
25
26 #mostrar información en el log
27 spring.jpa.properties.hibernate.format_sql=true
28 logging.level.org.hibernate.SQL=DEBUG
29 logging.level.org.type.descriptor.sql.BasicBinder=TRACE
30
```

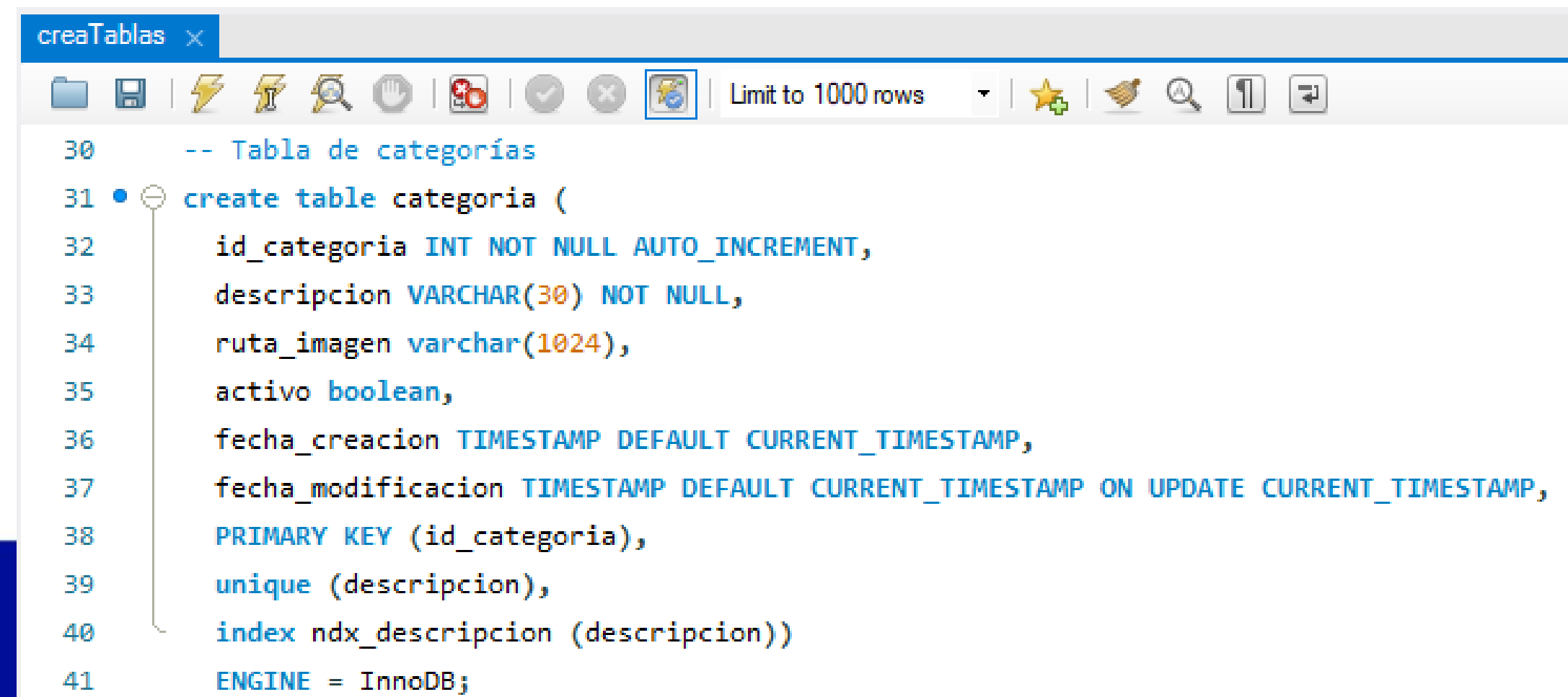

Patrón de diseño MVC



categoria

Para asociar una entidad en el proyecto que mapea la tabla categoría, se requiere analizar los atributos de la tabla y así traducirlo a una clase.

Así la clase “mapea” la tabla “categoria” de la base de datos.

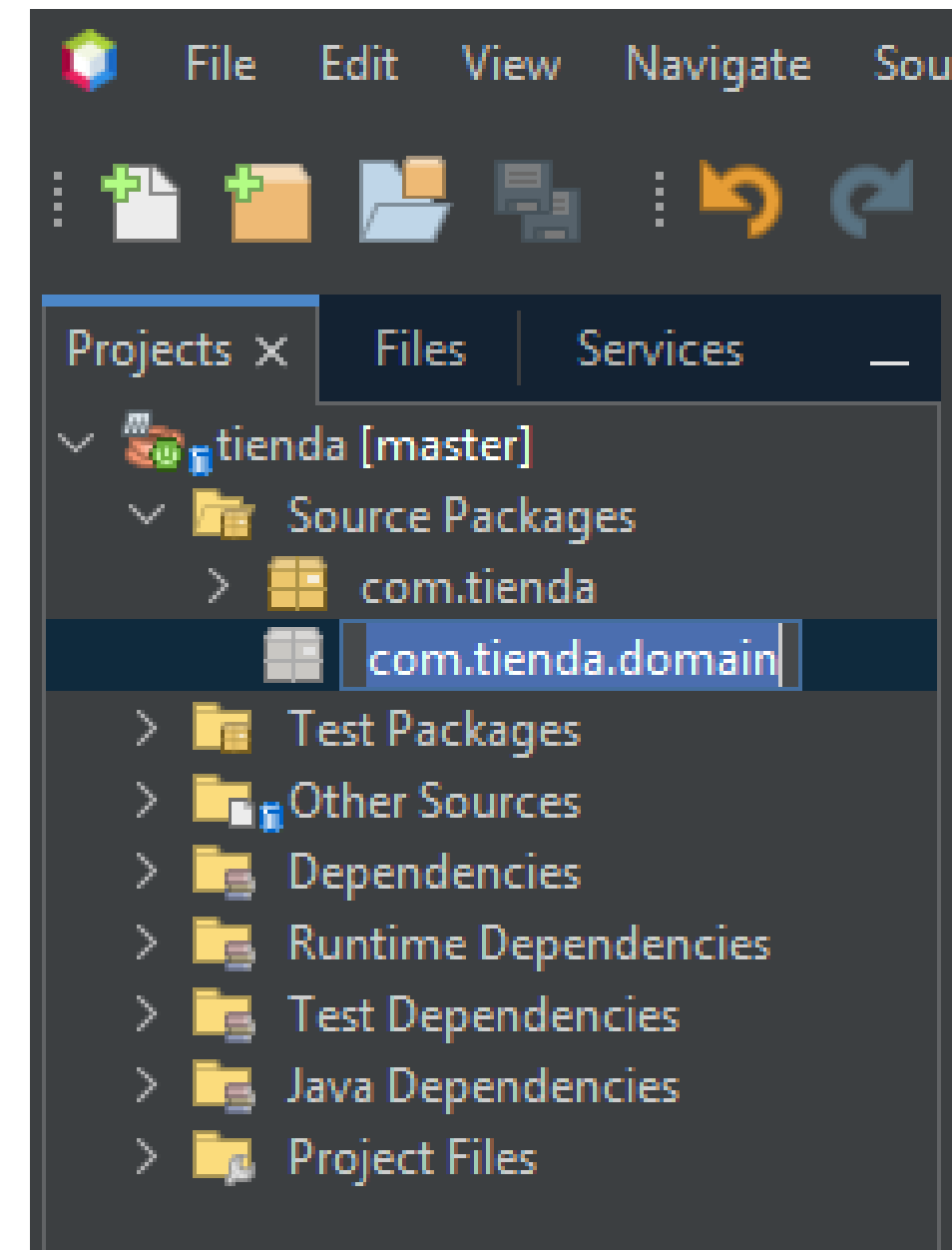


```
30  -- Tabla de categorías
31  create table categoria (
32      id_categoria INT NOT NULL AUTO_INCREMENT,
33      descripcion VARCHAR(30) NOT NULL,
34      ruta_imagen varchar(1024),
35      activo boolean,
36      fecha_creacion TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
37      fecha_modificacion TIMESTAMP DEFAULT CURRENT_TIMESTAMP ON UPDATE CURRENT_TIMESTAMP,
38      PRIMARY KEY (id_categoria),
39      unique (descripcion),
40      index ndx_descripcion (descripcion))
41  ENGINE = InnoDB;
```

domain

Se crea un “subpaquete” dentro del paquete principal “com.tienda”, llamado “domain”.

En este paquete se crearán todas las entidades del proyecto que “mapean” tablas de la base de datos.



Categoria.java

La clase tiene múltiples anotaciones importantes:

@Data

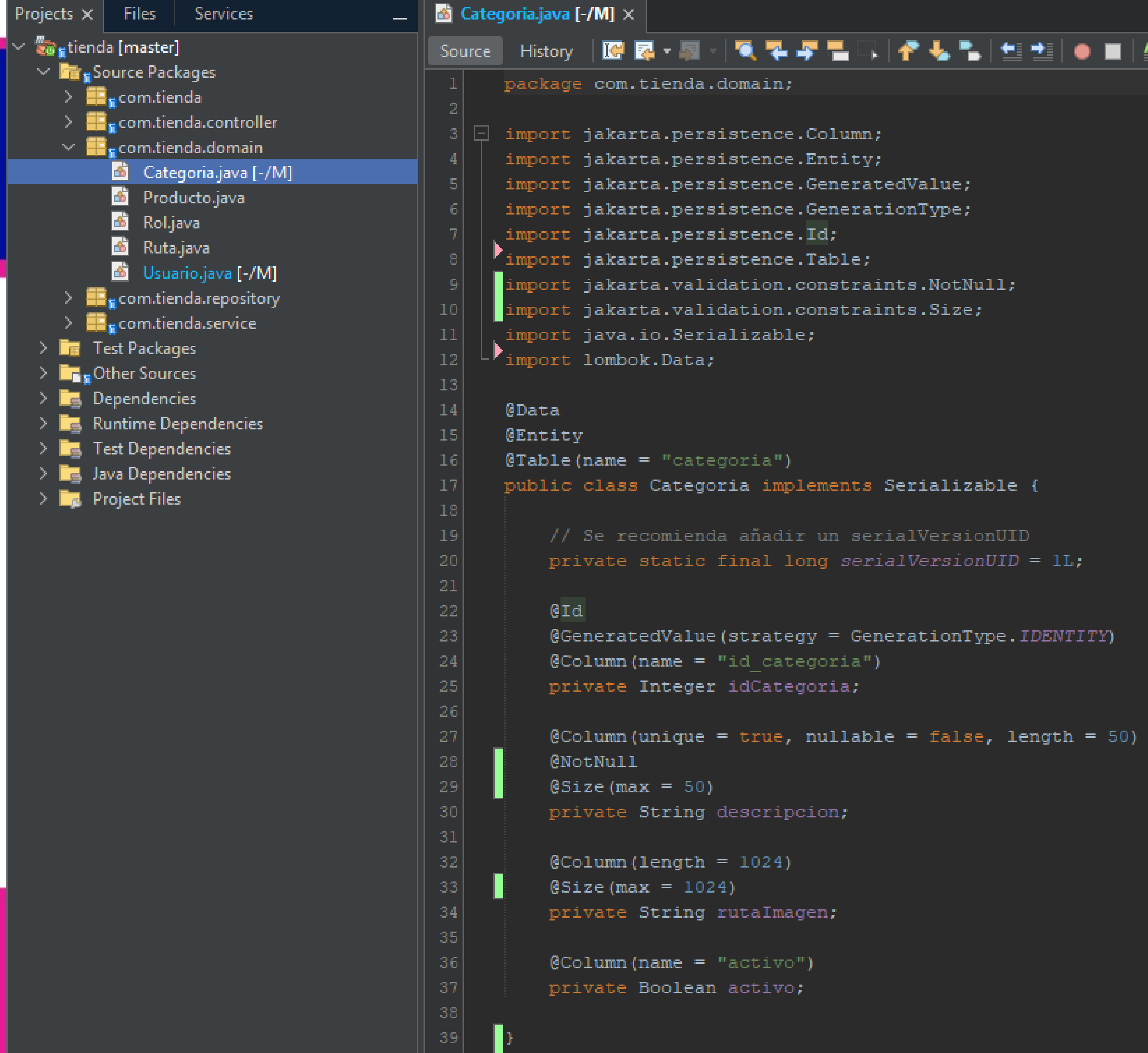
@Entity

@Table

@Id

@GeneratedValue

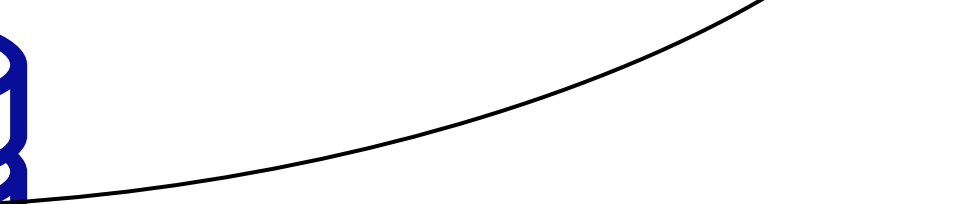
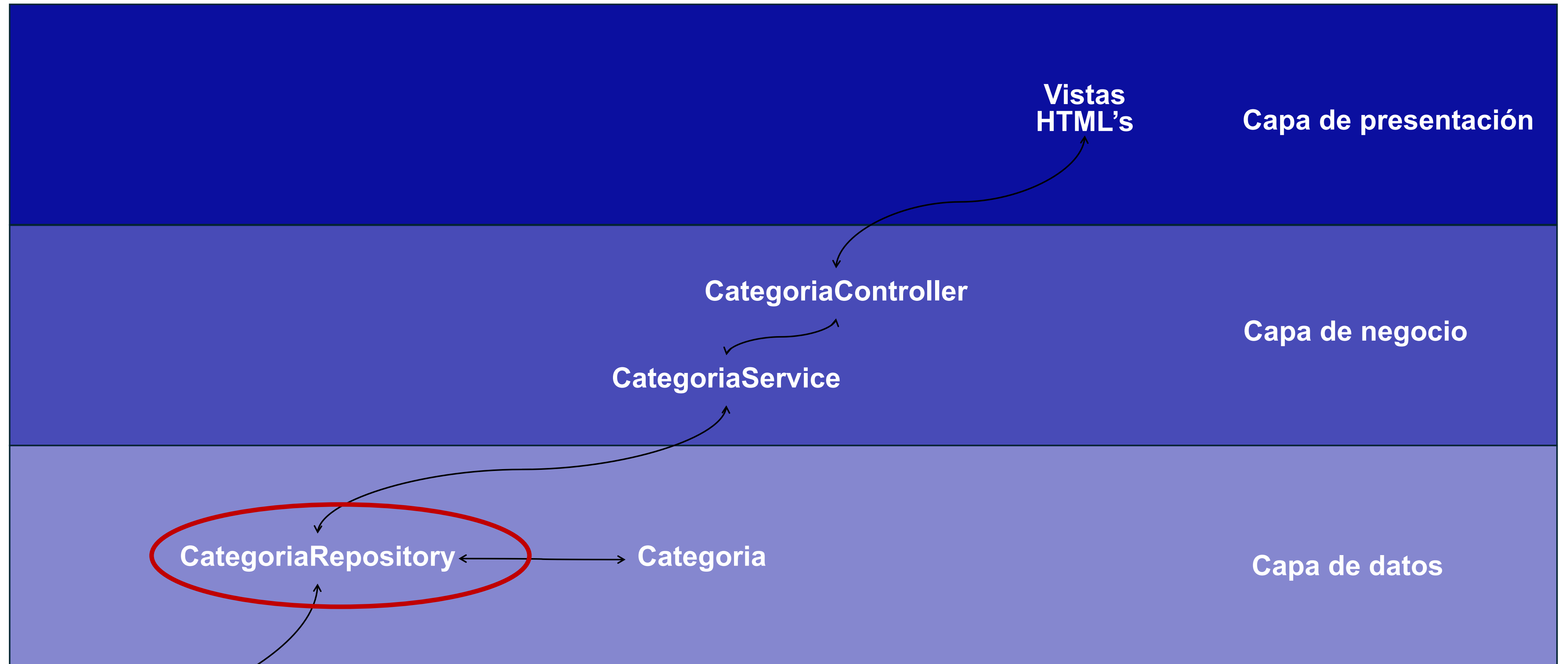
@Column



The screenshot shows an IDE with two panels. The left panel displays the project structure for 'tienda [master]', with 'com.tienda.domain.Categoria.java [-/M]' selected. The right panel shows the source code of 'Categoria.java'.

```
1 package com.tienda.domain;
2
3 import jakarta.persistence.Column;
4 import jakarta.persistence.Entity;
5 import jakarta.persistence.GeneratedValue;
6 import jakarta.persistence.GenerationType;
7 import jakarta.persistence.Id;
8 import jakarta.persistence.Table;
9 import jakarta.validation.constraints.NotNull;
10 import jakarta.validation.constraints.Size;
11 import java.io.Serializable;
12 import lombok.Data;
13
14 @Data
15 @Entity
16 @Table(name = "categoria")
17 public class Categoria implements Serializable {
18
19     // Se recomienda añadir un serialVersionUID
20     private static final long serialVersionUID = 1L;
21
22     @Id
23     @GeneratedValue(strategy = GenerationType.IDENTITY)
24     @Column(name = "id_categoria")
25     private Integer idCategoria;
26
27     @Column(unique = true, nullable = false, length = 50)
28     @NotNull
29     @Size(max = 50)
30     private String descripcion;
31
32     @Column(length = 1024)
33     @Size(max = 1024)
34     private String rutaImagen;
35
36     @Column(name = "activo")
37     private Boolean activo;
38
39 }
```

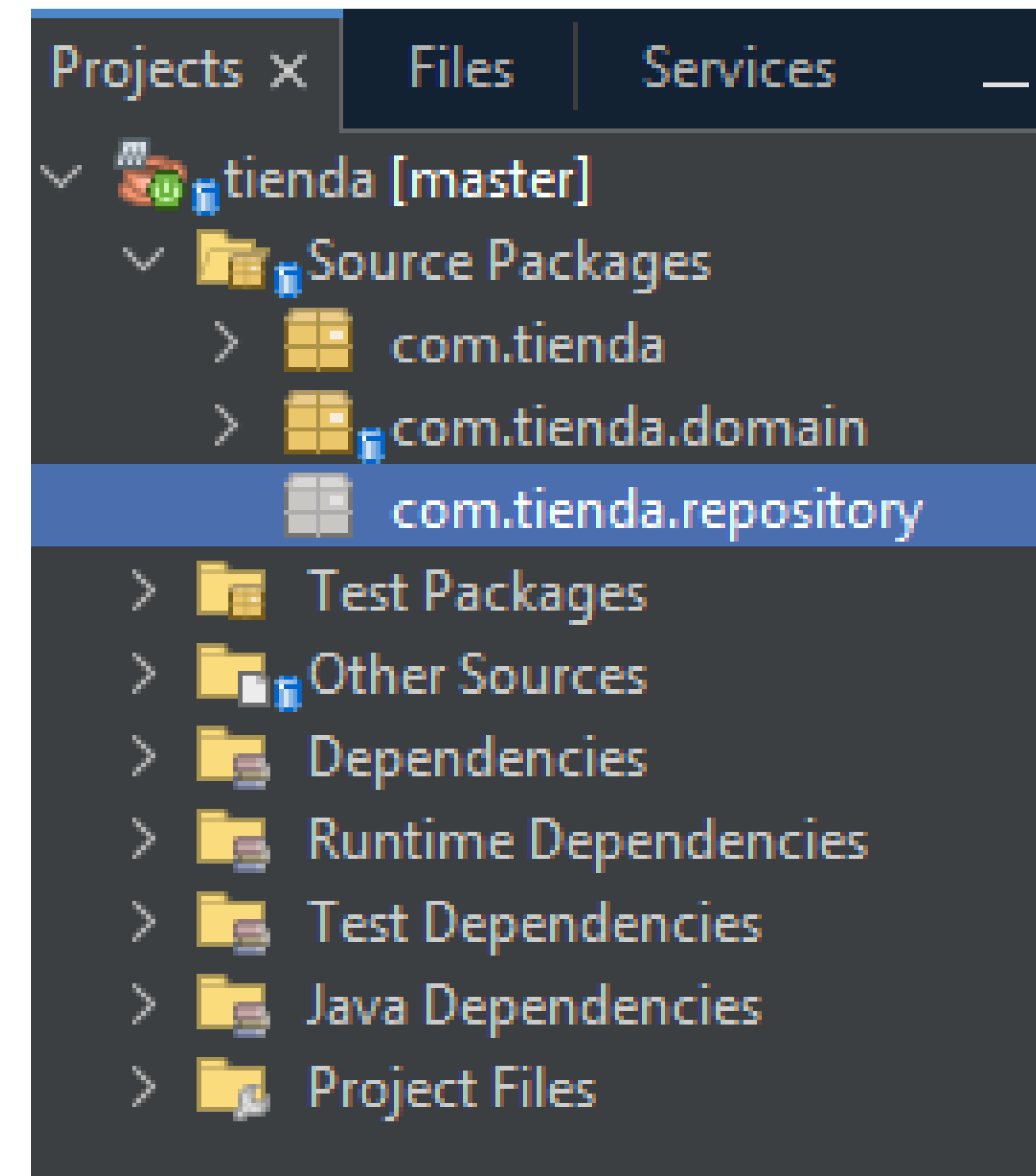

Patrón de diseño MVC



Package Repository

Se crea un “subpaquete” dentro del paquete principal “com.tienda”, llamado “repository”.

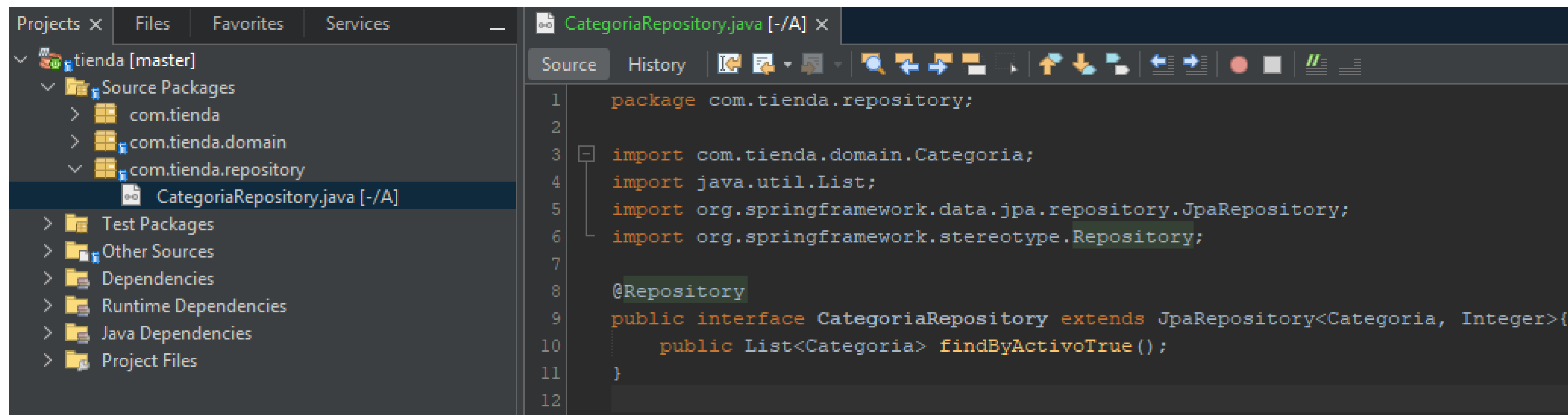
En este paquete se crearán todas las interfaces de acceso a datos del proyecto tienda.



CategoriaRepository.java

Dentro del paquete repository se crea la interface CategoriaRepository.java

Note que es una interface, no una clase. La interface hereda de JpaRepository y recibe objetos de tipo Categoria, que tiene como clave un elemento tipo Long.



The screenshot shows an IDE with a project named 'tienda [master]'. The 'Source Packages' view on the left shows the package structure: 'com.tienda' containing 'com.tienda.domain' and 'com.tienda.repository'. The 'CategoriaRepository.java' file is selected in the 'com.tienda.repository' package. The main editor window displays the following code:

```
1 package com.tienda.repository;
2
3 import com.tienda.domain.Categoria;
4 import java.util.List;
5 import org.springframework.data.jpa.repository.JpaRepository;
6 import org.springframework.stereotype.Repository;
7
8 @Repository
9 public interface CategoriaRepository extends JpaRepository<Categoria, Integer>{
10     public List<Categoria> findByActivoTrue();
11 }
12
```


Estructuras y Paquetes

1

Domain

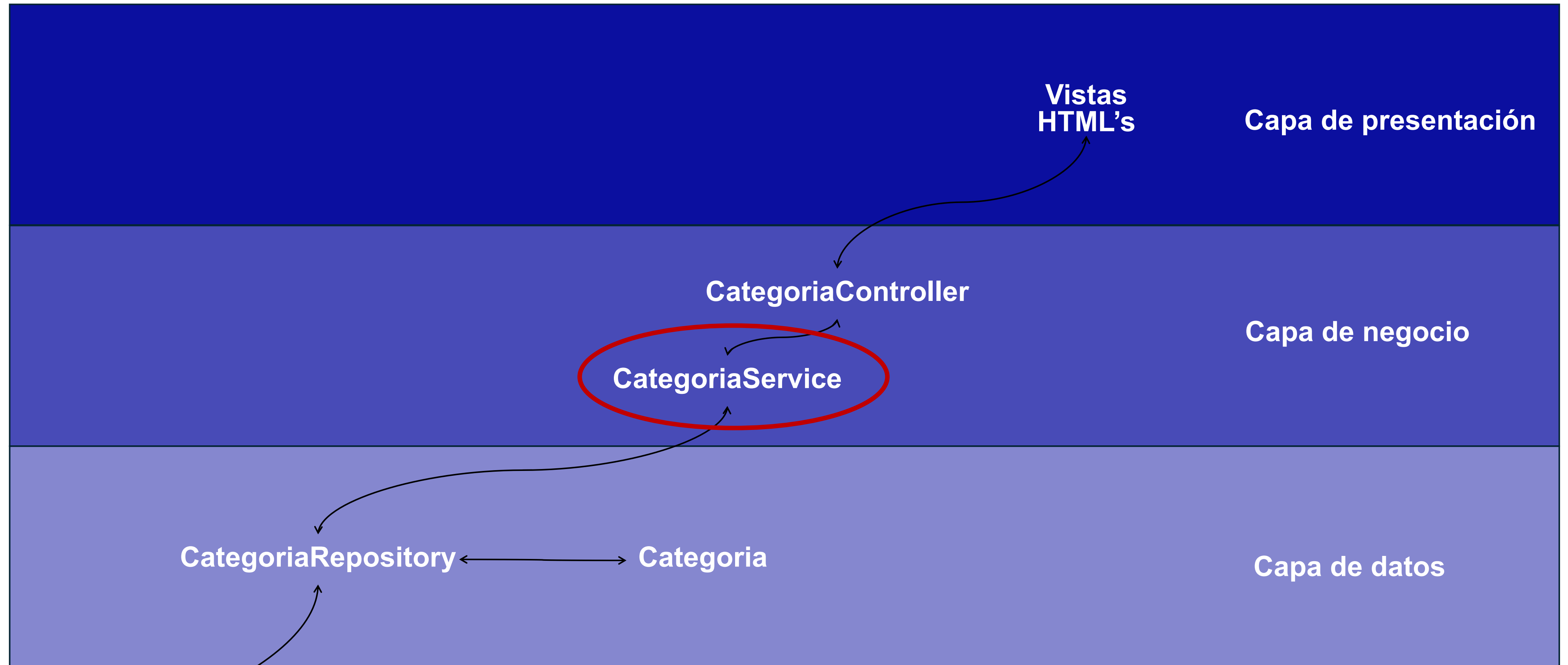
Donde se aloja la clase Categoria.java que mapea la tabla categoría de la base de datos.

2

Repository

Donde se creó la interface CategoriaRepository.java que implementa JpaRepository y esta implementación recibe objetos Categoria, así como id tipo Integer.

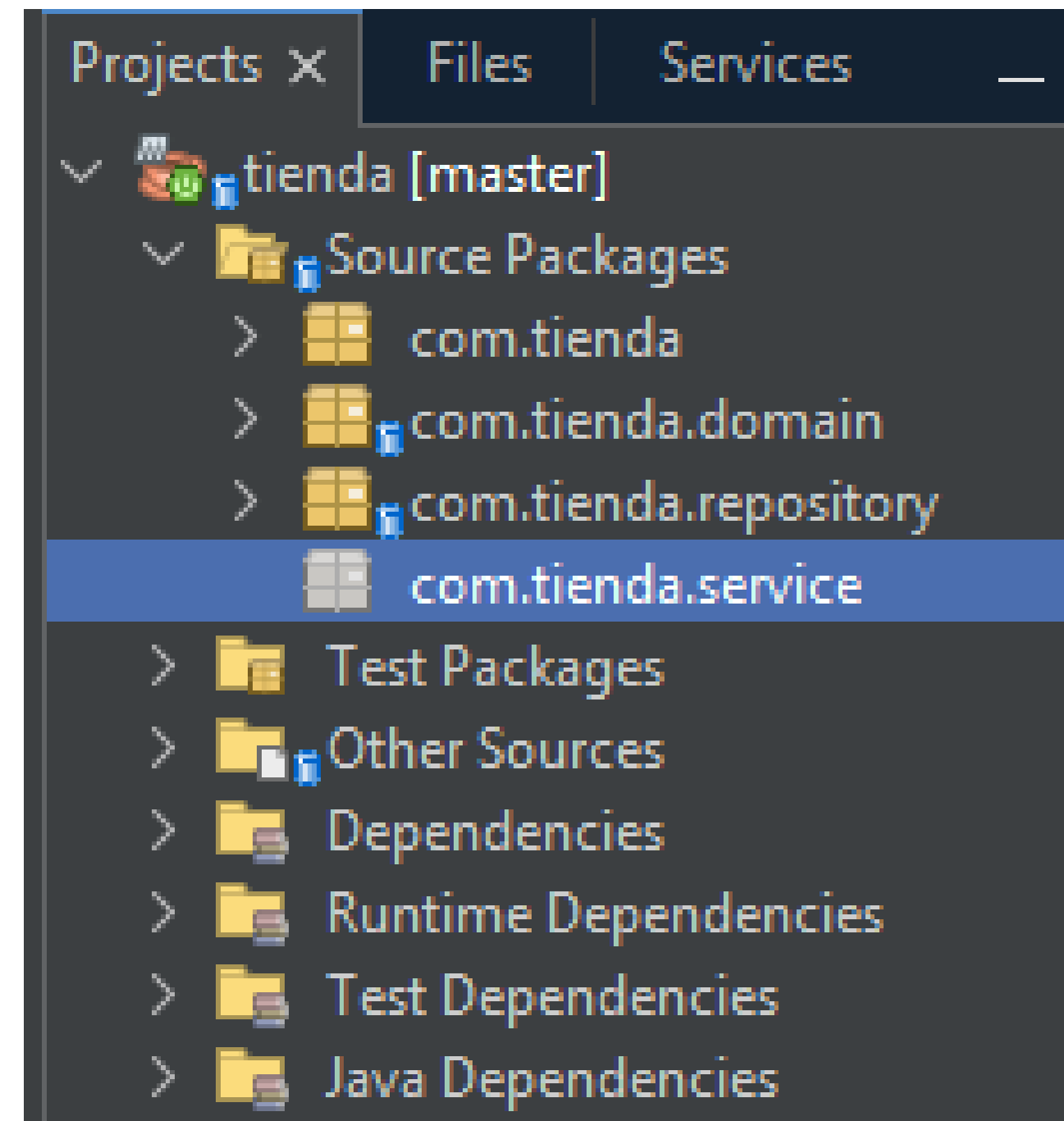
Patrón de diseño MVC



Paquete service

Se crea un “subpaquete” dentro del paquete principal “com.tienda”, llamado “service”.

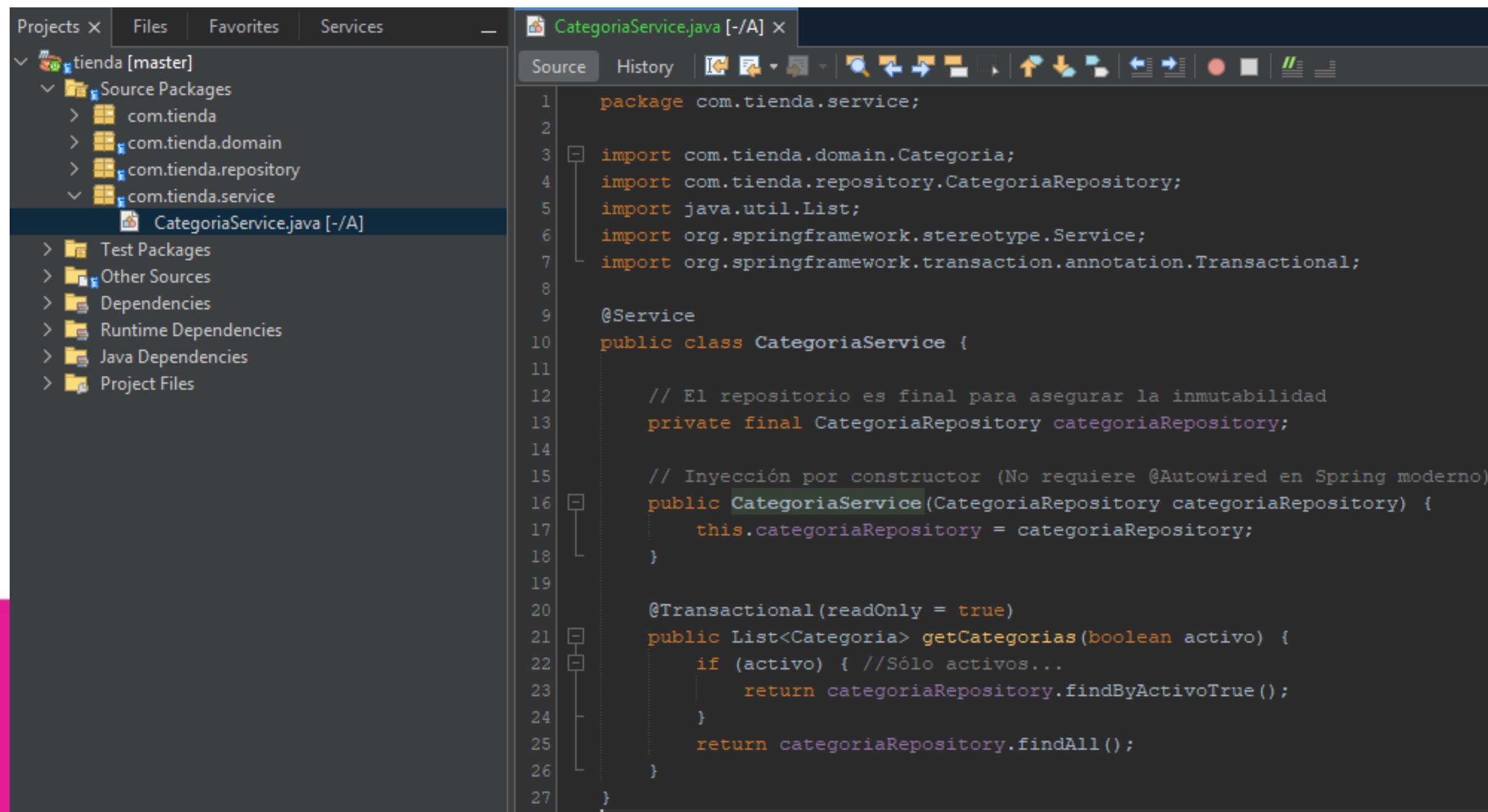
En este paquete se crearán todas las clases de servicio.



CategoriaService.java

La clase CategoriaService.java incorpora el enunciado del método getCategoryas.

Nótese que recibe un parámetro booleano para devolver o la lista completa o solo la lista de categorías activas.



```
1 package com.tienda.service;
2
3 import com.tienda.domain.Categoria;
4 import com.tienda.repository.CategoriaRepository;
5 import java.util.List;
6 import org.springframework.stereotype.Service;
7 import org.springframework.transaction.annotation.Transactional;
8
9 @Service
10 public class CategoriaService {
11
12     // El repositorio es final para asegurar la inmutabilidad
13     private final CategoriaRepository categoriaRepository;
14
15     // Inyección por constructor (No requiere @Autowired en Spring moderno)
16     public CategoriaService(CategoriaRepository categoriaRepository) {
17         this.categoriaRepository = categoriaRepository;
18     }
19
20     @Transactional(readOnly = true)
21     public List<Categoria> getCategoryas(boolean activo) {
22         if (activo) { //Sólo activos...
23             return categoriaRepository.findByActivoTrue();
24         }
25         return categoriaRepository.findAll();
26     }
27 }
```


Estructuras y Paquetes

1

Domain

Donde se aloja la clase Categoria.java que mapea la tabla categoría de la base de datos.

2

Repository

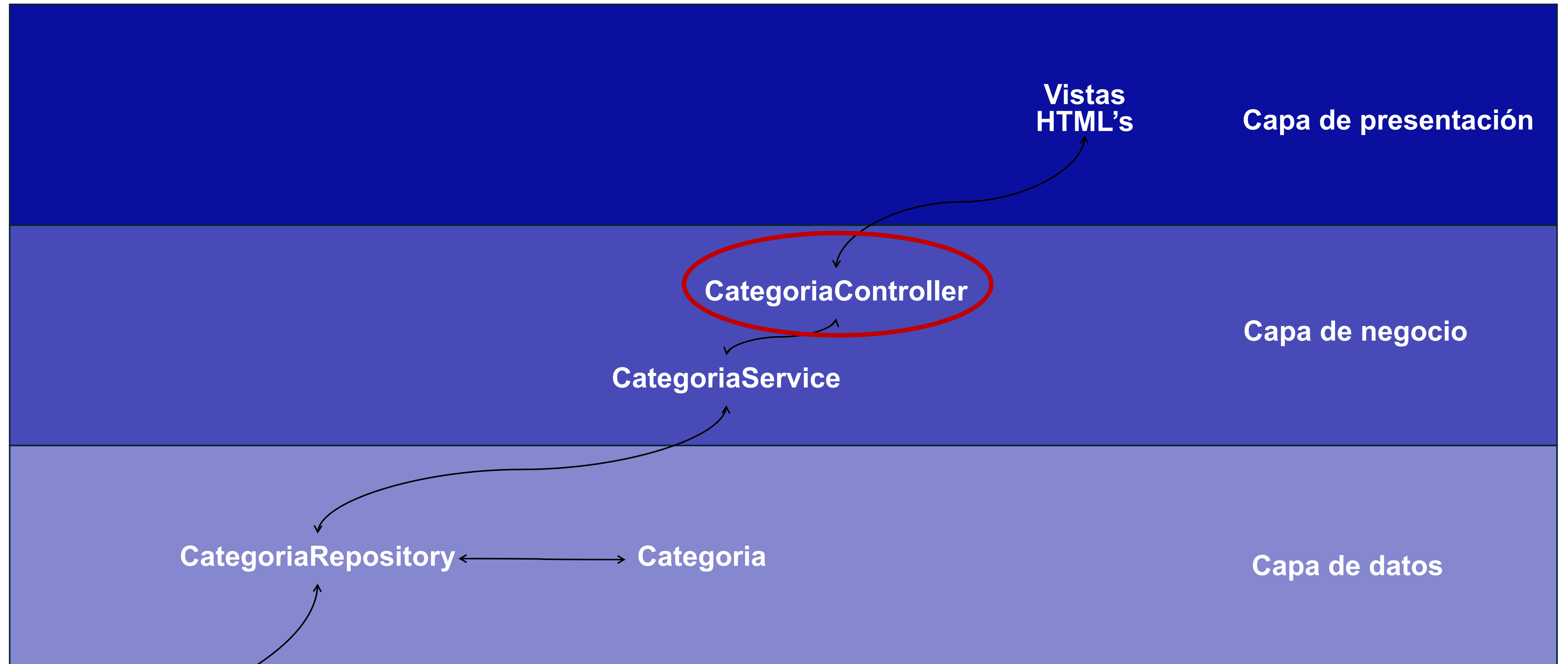
Donde se creó la interface CategoriaRepository.java que implementa JpaRepository y esta implementación recibe objetos Categoria, así como id tipo Integer.

3

Service

La clase CategoriaService.java donde básicamente se utiliza el método “findAll()” de JpaRepository.

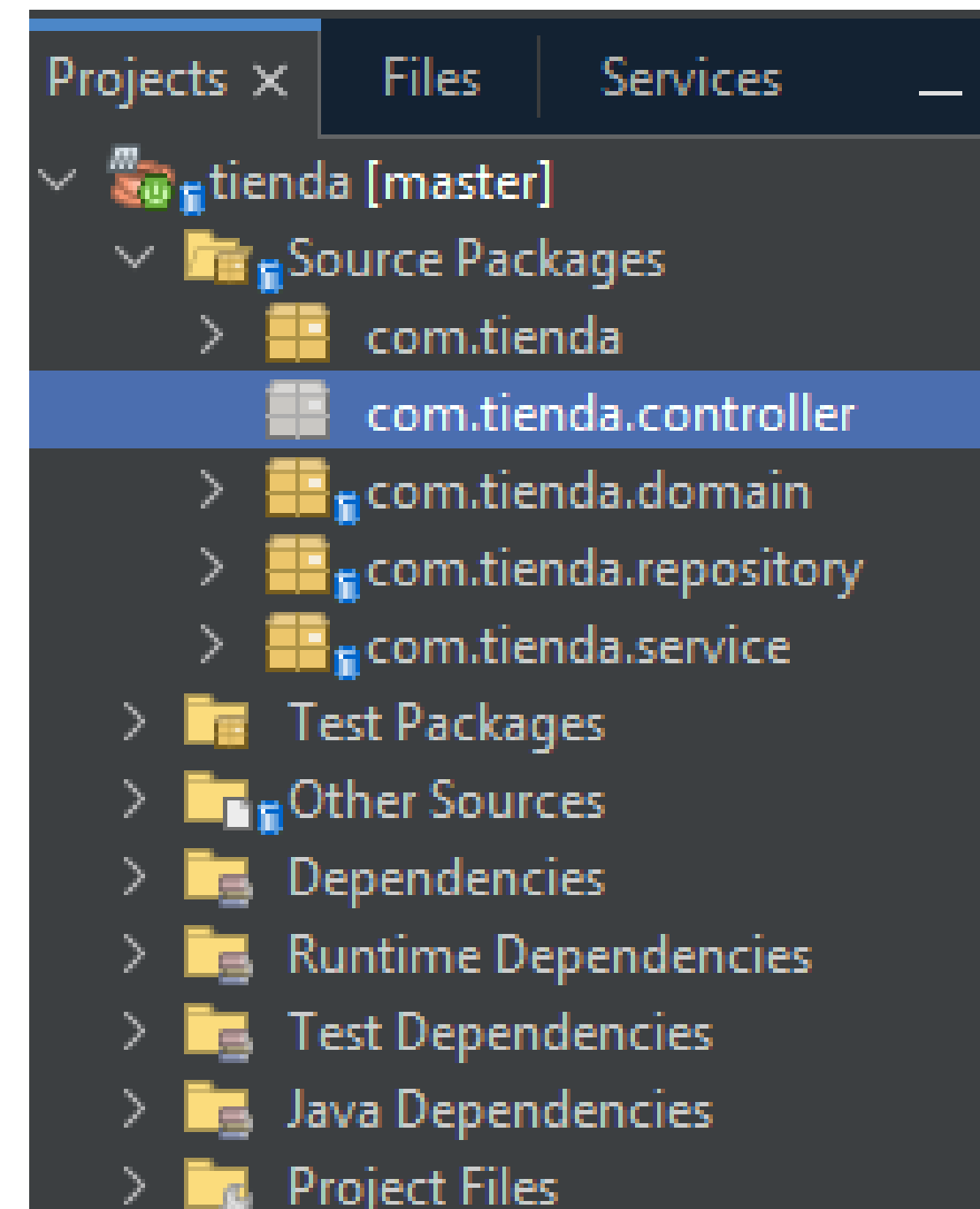
Patrón de diseño MVC



Paquete controller

Se crea un “subpaquete” dentro del paquete principal “com.tienda”, llamado “controller”.

En este paquete se crearán todos los controladores del proyecto que ejecutan las “rutas” solicitadas en las diferentes vistas del proyecto.



CategoriaController.java

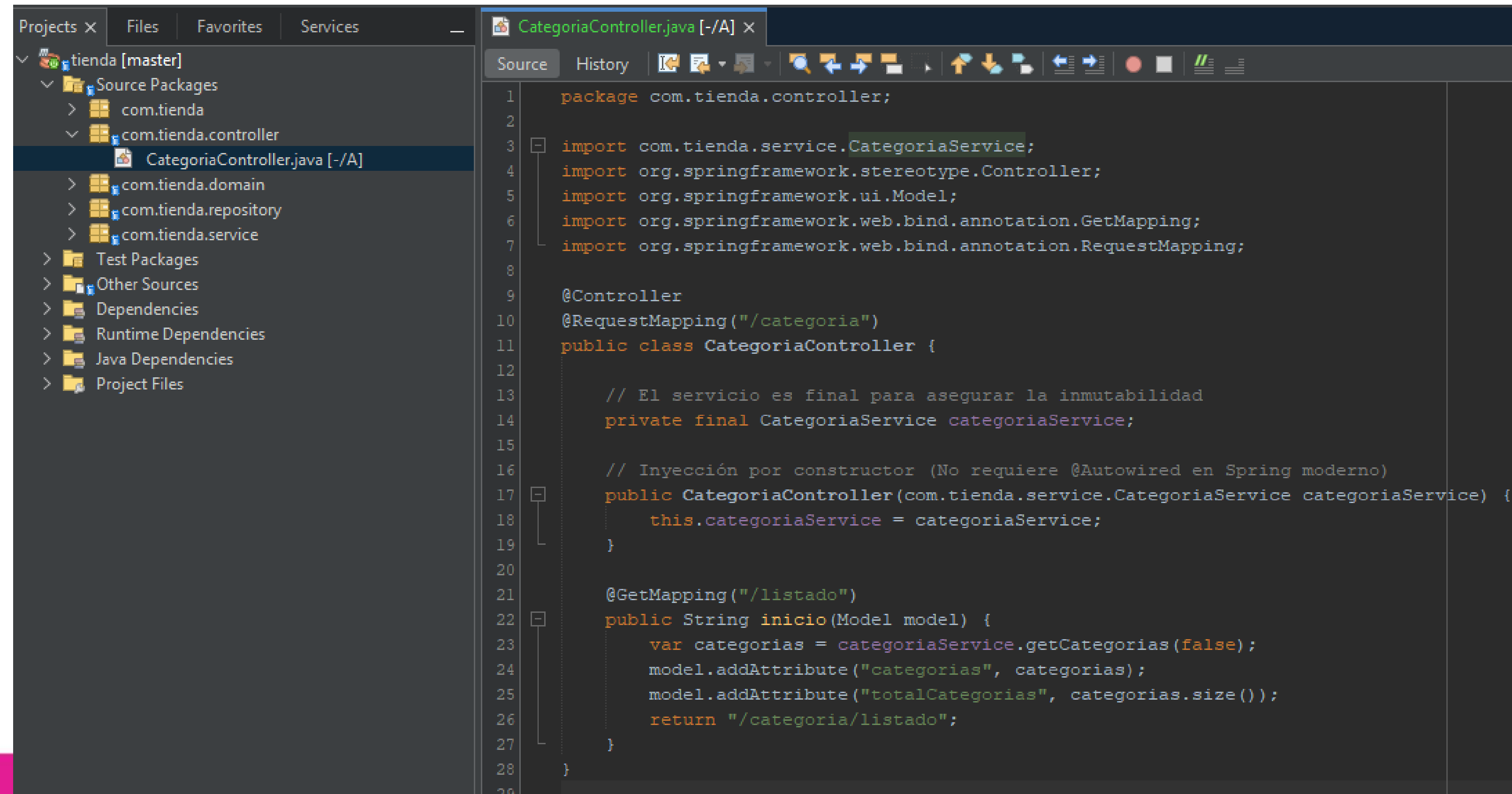
La clase CategoríaController.java

Nótese el uso de las anotaciones:

@Controller

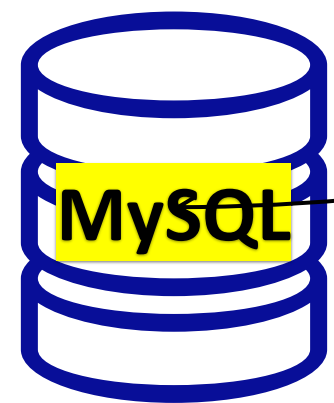
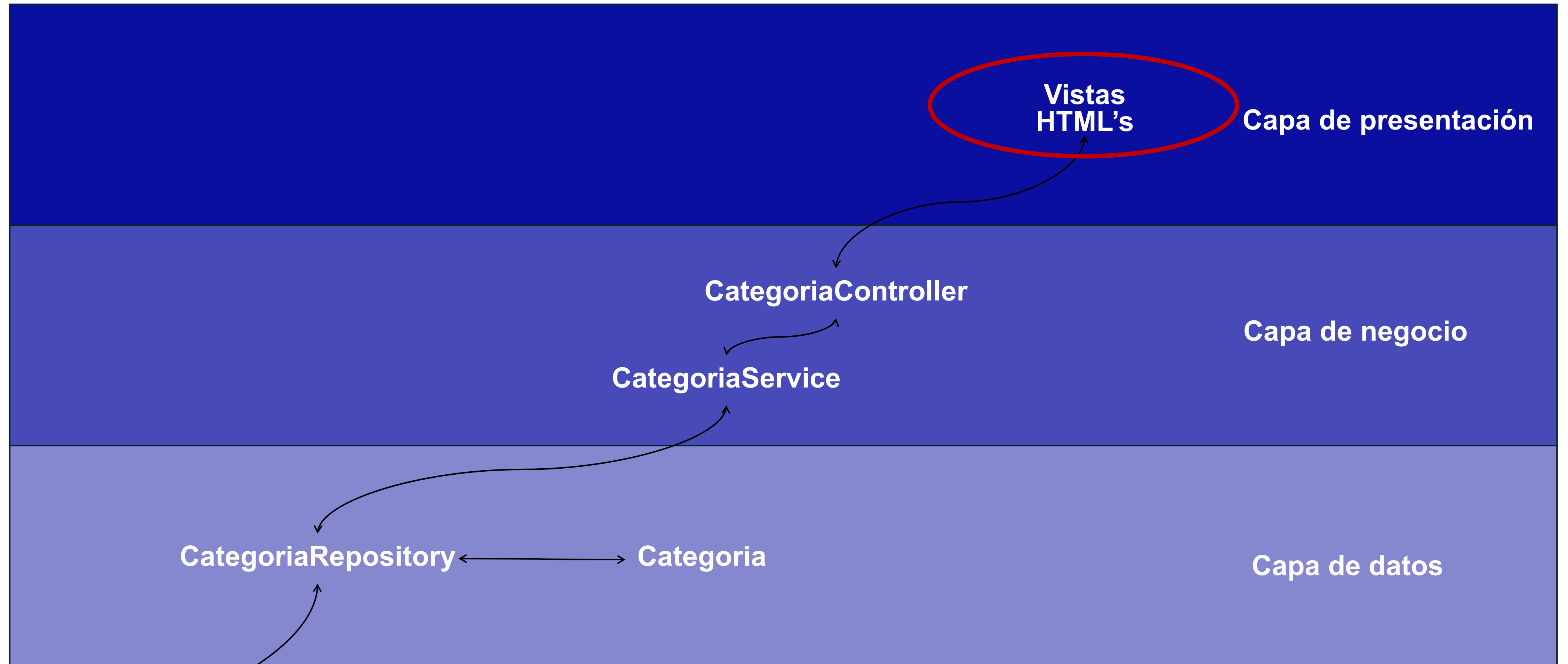
@RequestMapping

@GetMapping



```
1 package com.tienda.controller;
2
3 import com.tienda.service.CategoriaService;
4 import org.springframework.stereotype.Controller;
5 import org.springframework.ui.Model;
6 import org.springframework.web.bind.annotation.GetMapping;
7 import org.springframework.web.bind.annotation.RequestMapping;
8
9 @Controller
10 @RequestMapping("/categoria")
11 public class CategoriaController {
12
13     // El servicio es final para asegurar la inmutabilidad
14     private final CategoriaService categoriaService;
15
16     // Inyección por constructor (No requiere @Autowired en Spring moderno)
17     public CategoriaController(com.tienda.service.CategoriaService categoriaService) {
18         this.categoriaService = categoriaService;
19     }
20
21     @GetMapping("/listado")
22     public String inicio(Model model) {
23         var categorias = categoriaService.getCategorias(false);
24         model.addAttribute("categorias", categorias);
25         model.addAttribute("totalCategorias", categorias.size());
26         return "/categoria/listado";
27     }
28 }
29
```

Patrón de diseño MVC



MySQL

La ruta en el browser

http://localhost:xxxx/categoría/listado

Protocolo

Equipo/ip/dominio

Puerto

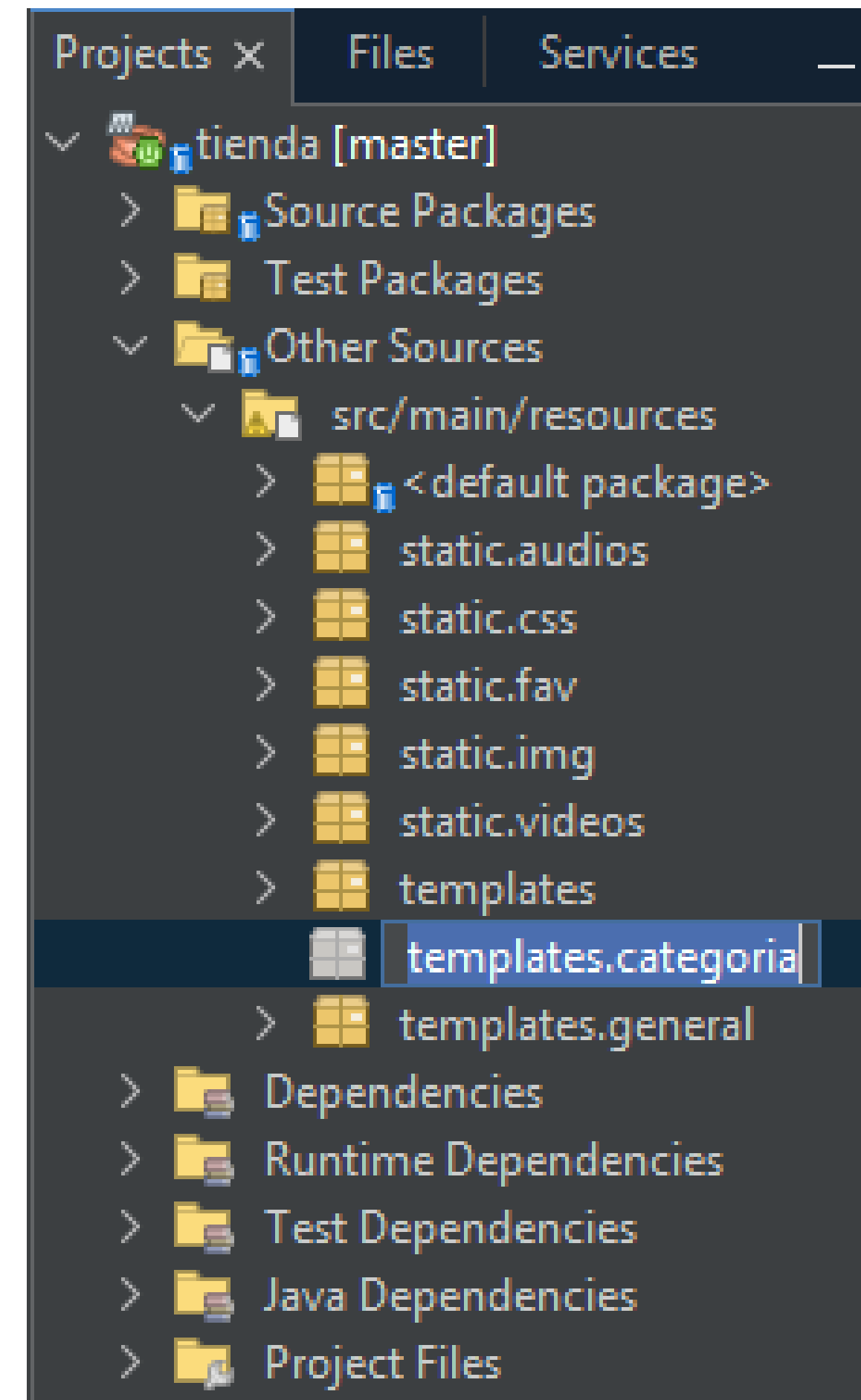
RequestMapping

GetMapping

Folder categoria

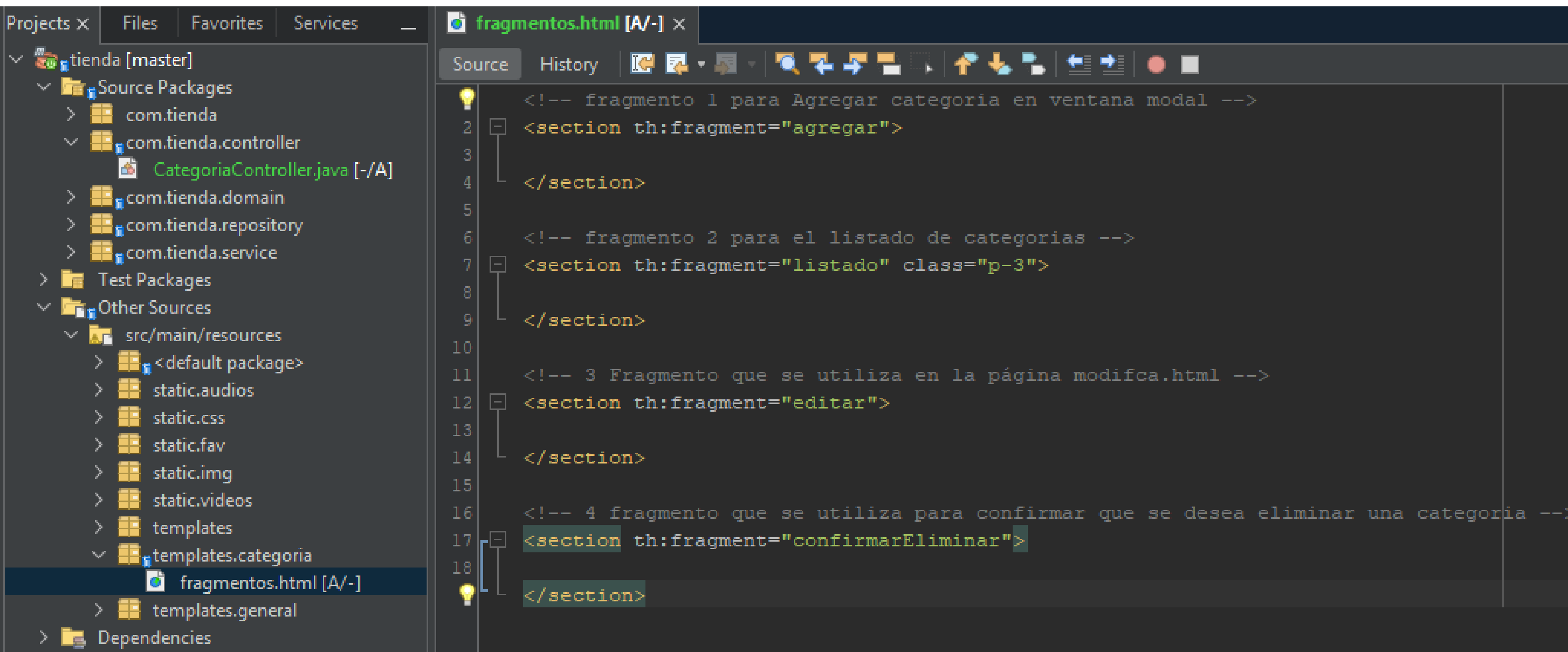
Se crea un “subfolder” dentro del folder principal “templates”, llamado “categoria”.

En este folder se crearán todas las vistas relacionadas con el manejo de categorías del proyecto.



fragmentos.html

En este archivo se crean los fragmentos a utilizar para categoria



The screenshot shows an IDE with a project named 'tienda [master]'. The file explorer on the left shows the project structure, including 'Source Packages' (com.tienda, com.tienda.controller, com.tienda.domain, com.tienda.repository, com.tienda.service), 'Test Packages', and 'Other Sources' (src/main/resources). The 'fragmentos.html' file is selected in the 'src/main/resources' directory. The main editor displays the content of 'fragmentos.html', which contains four HTML fragments for a category management application.

```
1 <!-- fragmento 1 para Agregar categoria en ventana modal -->
2 <section th:fragment="agregar">
3
4 </section>
5
6 <!-- fragmento 2 para el listado de categorias -->
7 <section th:fragment="listado" class="p-3">
8
9 </section>
10
11 <!-- 3 Fragmento que se utiliza en la página modifca.html -->
12 <section th:fragment="editar">
13
14 </section>
15
16 <!-- 4 fragmento que se utiliza para confirmar que se desea eliminar una categoria -->
17 <section th:fragment="confirmarEliminar">
18
19 </section>
```

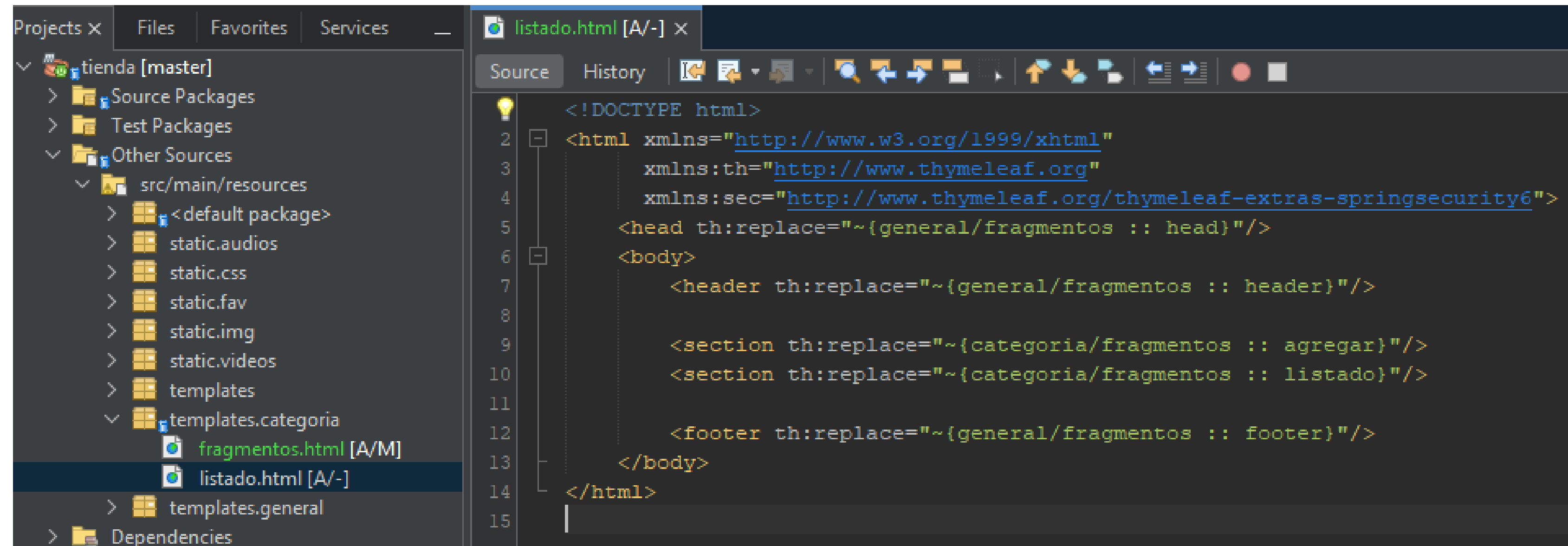

fragmentos.html

En este archivo se crean los fragmentos a utilizar para categoria

```
1 <!-- fragmento 1 para Agregar categoria en ventana modal -->
2 <section th:fragment="agregar">
3     <!-- Button trigger modal -->
4     <div class='row py-4 mb-2 bg-light'>
5         <div class='col-md-1'></div>
6         <div class='col-md-3'>
7             <button type="button" class="btn btn-primary" sec:authorize="hasRole('ROLE_ADMIN')"
8                 data-bs-toggle="modal" data-bs-target="#agregarCategoriaModal">
9                 <i class="fa-solid fa-plus"></i> [[#{categoria.agregar}]]</button>
10        </div>
11    </div>
12    <!-- Modal -->
13    <div class="modal fade" id="agregarCategoriaModal" tabindex="-1"
14        aria-labelledby="exampleModalLabel" aria-hidden="true">
15        <div class="modal-dialog">
16            <div class="modal-content">
17                <div class="modal-header bg-info text-white">
18                    <h1 class="modal-title fs-5" id="exampleModalLabel">
19                        <i class="fa-solid fa-layer-group"></i> [[#{categoria.agregar}]]</h1>
20                    <button type="button" class="btn-close" data-bs-dismiss="modal" aria-label="Close"></button>
21                </div>
22                <form th:action="@{/categoria/guardar}" th:object="${categoria}"
23                    class='was-validated' enctype="multipart/form-data" method="post">
24                    <div class="modal-body">
25                        <div class='mb-3'>
26                            <label>[[#{categoria.descripcion}]]</label>
27                            <input type="text" name='descripcion' class='form-control' required="true"/>
28                        <div class='mb-3'>
29                            <label>[[#{categoria.activo}]]</label>
30                            <input type="checkbox" name='activo' class='form-check-input'/>
31                        <div class='mb-3'>
32                            <label>[[#{categoria.imagen}]]</label>
33                            <input type="file" name='imagenFile' onchange='mostrarImagen(this);'
34                                class='form-control'/>
35                            <img id='blah' src='#' alt='imagen de categoria' height="200"/>
36                        </div>
37                        <div class="modal-footer">
38                            <button type="button" class="btn btn-secondary" data-bs-dismiss="modal">
39                                <i class="fa-solid fa-xmark"></i> [[#{accion.cerrar}]]</button>
40                            <button type="submit" class="btn btn-primary">
41                                <i class="fa-solid fa-floppy-disk"></i> [[#{accion.guardar}]]</button>
42                        </div>
43                    </form>
44                </div>
45            </div>
46        </div>
47    </section>
48
```

listado.html

En este archivo se crea el llamado a los fragmentos



The screenshot shows an IDE with a project named 'tienda [master]'. The file explorer on the left shows the following structure:

- tienda [master]
 - Source Packages
 - Test Packages
 - Other Sources
 - src/main/resources
 - <default package>
 - static.audios
 - static.css
 - static.fav
 - static.img
 - static.videos
 - templates
 - templates.categoria
 - fragmentos.html [A/M]
 - listado.html [A/-]
 - templates.general
 - Dependencies

The main editor shows the content of 'listado.html' [A/-] with the following code:

```
<!--DOCTYPE html-->
<html xmlns="http://www.w3.org/1999/xhtml"
      xmlns:th="http://www.thymeleaf.org"
      xmlns:sec="http://www.thymeleaf.org/thymeleaf-extras-springsecurity6">
  <head th:replace="~{general/fragmentos :: head}"/>
  <body>
    <header th:replace="~{general/fragmentos :: header}"/>
    <section th:replace="~{categoria/fragmentos :: agregar}"/>
    <section th:replace="~{categoria/fragmentos :: listado}"/>
    <footer th:replace="~{general/fragmentos :: footer}"/>
  </body>
</html>
```

Así se ve la salida

← → ↺

localhost/categoria/listado

🔍 ☆ 🔒 🏠

Verifica que eres tú

Herramientas IA | aprendizaje | Biblioteca de Cursos

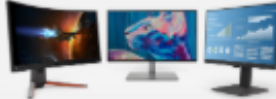
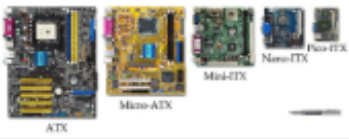
Todos los f

TechShop | Categorías | Productos | Consultas | Administración ▾ | Idioma ▾

Su tienda en línea

+ Agregar Categoría

Listado de Categorías

#	Descripción	Imagen	Activo	
1	Monitores		Activa	Eliminar Actualizar
2	Teclados		Activa	Eliminar Actualizar
3	Tarjeta Madre		Activa	Eliminar Actualizar
4	Celulares		Inactiva	Eliminar Actualizar

Total Categorías
4

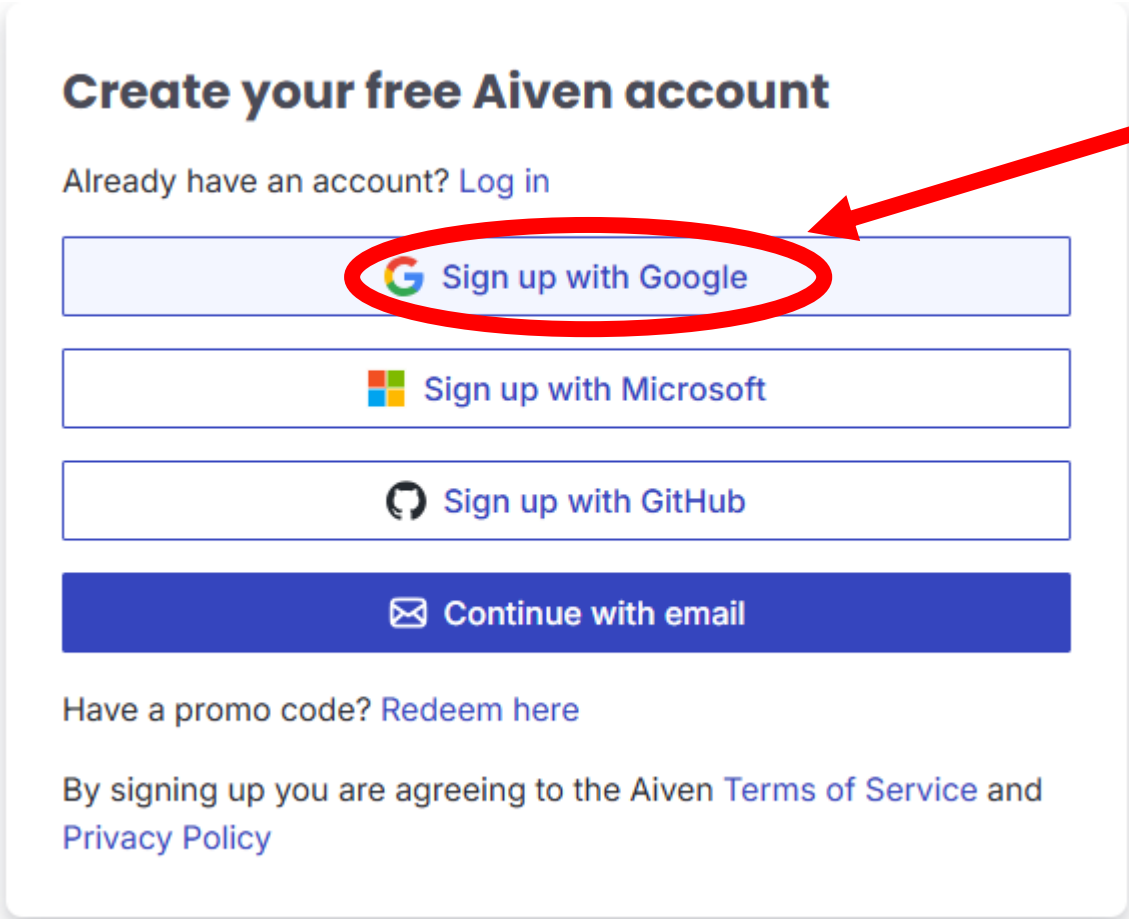
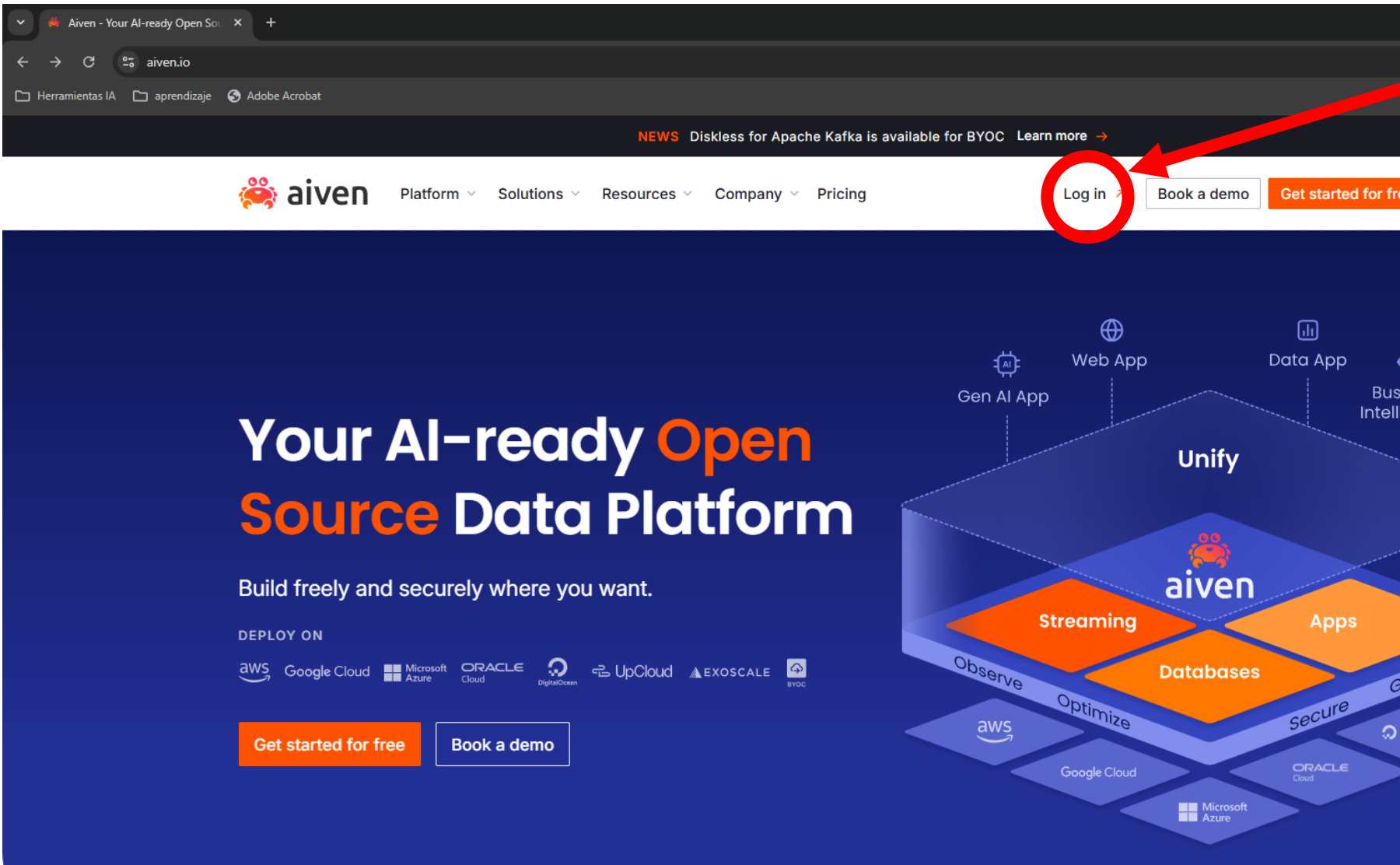
Nos vemos en semana 4

Reto: el proyecto en render.com deja de funcionar porque introducimos una base de datos local que no está en render. ¿Cómo hacemos para que funcione?

Tip: crea una cuenta en aiven.com, crea una base de datos MySQL, haz una conexión en WorkBench, ejecuta el creaTablas.sql. Modifica application.properties para usar esa base de datos en la nube. Con esto ahora si funcionará el nuevo commit que hagas.

Creando la cuenta en aiven.io

<https://aiven.io/>



Dentro de aiven.io

Services | Aiven

console.aiven.io/account/a54b4fec4b5a/project/techshop-40159/services

Herramientas IA | aprendizaje | Adobe Acrobat

aiven
CONSOLE

Home Projects Tools Billing Admin

PROJECT
techshop-40159

My Organization / techshop-40159 / Create new service

Create new service

- Get started
- Services
- Integration endpoints
- VPCs
- Event log
- Permissions
- Project AI insights
- Settings

- AlloyDB Omni
AlloyDB Omni - High-performance PostgreSQL-compatible database with AI capabilities
- MySQL**
MySQL - Popular general-purpose easy-to-use relational database
- ClickHouse®
ClickHouse - Fast resource-effective data warehouse for analytical workloads
- Grafana®
Grafana - Data visualization and analytics platform

Home | Aiven

console.aiven.io/account/a54b4fec4b5a/home

Herramientas IA | aprendizaje | Adobe Acrobat

aiven
CONSOLE

Home Projects Tools Billing Admin

Optimize ClickHouse query performance with tiered storage

Unlock cost-efficient storage for your ClickHouse datasets. Tiered storage in Aiven for ClickHouse automatically moves cold data to remote storage, reducing your infrastructure footprint and ensuring your analytical queries remain lightning-fast.

[Learn more →](#)

Create your first project

Projects are used to group related services together. Create a project to get started with Aiven services.

Create project

Create project

Project name*
techshop

Organization or organizational unit*
My Organization

Billing group*
Billing group for My Organization

[Add tag](#)

[Cancel](#) **Create project**

Definiendo información

Create service

Project: techshop-40159 Organization: My Organization

< Back

MySQL

MySQL - Popular general-purpose easy-to-use relational database

Name*

The service name cannot be changed afterwards

techshop

+ Add tag to this service

Select plan type

Free

Free plan

- One free managed MySQL service
- Good for building a small application and learning to use cloud databases
- No credit card required

Full platform

- 30-day trial with \$300 of free usage included
- Good for proof of concepts and testing Aiven for production use cases
- No credit card required

Select region

You can select a specific cloud provider and region on paid plans.

Asia Pacific

Australia

Europe

North America

Service summary

Service

MySQL

Name

techshop

Region

North America

Plan

Free-1-1gb

@1 CPU @ 1 GB RAM @ 1 GB storage

Backups for disaster recovery

Monthly price

Free

Create service

My Organization / techshop-40159 / techshop / Overview

techshop

MySQL 8.0.35

Running

Nodes 1

fidÉlitas
Universidad

La sección de copia a workbeach

Connection information

Quick connect

MySQL

MySQLx

Service URI	mysql://CLICK_TO:REVEAL_PASSWORD@techshop-techshop-40159.i.aivencloud.com:24382/defaultdb?ssl-mode=REQUIRED	
Database name	defaultdb	
Host	techshop-techshop-40159.i.aivencloud.com	
Port	24382	
User	avnadmin	
Password	*****	

Setup New Connection

Connection Name: AivenNube

Type a name for the connection

Connection Method: Standard (TCP/IP)

Method to use to connect to the RDBMS

Parameters

SSL

Advanced

Hostname: jshop-techshop-40159.i.aivencoud.com

Port: 24382

Name or IP address of the server host - and TCP/IP port.

Username: avnadmin

Name of the user to connect with.

Password:

Store in Vault ...

Clear

The user's password. Will be requested later if it's not set.

Default Schema: defaultdb

The schema to use as default schema. Leave blank to select it later.

Configure Server Management...

Test Connection

Cancel

OK

fidÉlitas
Un i v e r s i d a d

Se ejecuta creaTablas en la nube

AivenNube

FileEditViewQueryDatabaseServerToolsScriptingHelp

Navigator

SCHEMAS

Filter objects

defaultdb
Tables
Views
Stored Procedures
Functions
sys

creaTablas

Limit to 1000 rows

```
70     ruta_imagen varchar(1024),
71     activo boolean,
72     fecha_creacion TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
73     fecha_modificacion TIMESTAMP DEFAULT CURRENT_TIMESTAMP ON UPDATE CURRENT_TIMESTAMP,
74     PRIMARY KEY (`id_usuario`),
75     CHECK (correo REGEXP '^[a-zA-Z0-9._%+-]+@[a-zA-Z0-9.-]+\.[a-zA-Z]{2,}$'),
76     index ndx_username (username))
77 ENGINE = InnoDB;
78
79 -- Tabla de roles
80 create table rol (
81     id_rol INT NOT NULL AUTO_INCREMENT,
82     rol varchar(20) unique,
83     fecha_creacion TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
84     fecha_modificacion TIMESTAMP DEFAULT CURRENT_TIMESTAMP ON UPDATE CURRENT_TIMESTAMP,
85     primary key (id_rol))
86 ENGINE = InnoDB;
87
88 -- Tabla de relación entre usuarios y roles
89 create table usuario_rol (
90     id_usuario int not null,
91     id_rol INT NOT NULL,
92     fecha_creacion TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
93     fecha_modificacion TIMESTAMP DEFAULT CURRENT_TIMESTAMP ON UPDATE CURRENT_TIMESTAMP,
94     PRIMARY KEY (id_usuario,id_rol),
95     foreign key fk_usuarioRol_usuario (id_usuario) references usuario(id_usuario),
96     foreign key fk_usuarioRol_rol (id_rol) references rol(id_rol))
97 ENGINE = InnoDB;
98
99
```

No object selected

Output

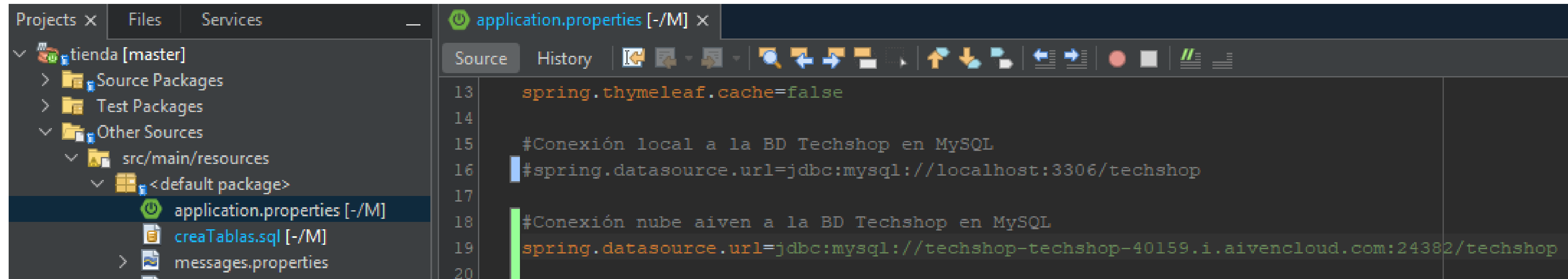
Action Output

#	Time	Action	Message
✓ 24	19:14:16	INSERT INTO venta (id_factura,id_producto,precio_historico,cantidad) values (1,5,45000.3), (1,9,15780.2), (1,10,15000.3), (2,5,45000.1), (2,14,154000....	18 row(s) affected Records: 18 Duplicates: 0 Warnings:
✓ 25	19:14:17	insert into rol (rol) values ('ADMIN'), ('VENDEDOR'), ('USER')	3 row(s) affected Records: 3 Duplicates: 0 Warnings:
✓ 26	19:14:17	insert into usuario_rol (id_usuario, id_rol) values (1,1), (1,2), (1,3),(2,2),(2,3),(3,3)	6 row(s) affected Records: 6 Duplicates: 0 Warnings:
✓ 27	19:14:17	INSERT INTO ruta (ruta, id_rol) VALUES ('producto/nuevo', 1), ('producto/guardar', 1), ('producto/modificar','', 1), ('producto/eliminar','', 1), ('cate...	19 row(s) affected Records: 19 Duplicates: 0 Warnings:
✓ 28	19:14:17	INSERT INTO ruta (ruta,requiere_rol) VALUES ('/false'), ('index' false), ('errores''false), ('camito''false), ('registro''false), ('/403'false), ('fav/'...	9 row(s) affected Records: 9 Duplicates: 0 Warnings:
✓ 29	19:14:17	INSERT INTO constante (atributo,valor) VALUES ('dominio',localhost'), ('dolar',520.75), ('url_paypal_cancel','http://localhost/payment/cancel'), ('url_pa...	4 row(s) affected Records: 4 Duplicates: 0 Warnings:

Object Info

Session

Los ajustes en application.properties



The screenshot shows an IDE window with the file explorer on the left and the code editor on the right. The file explorer shows a project named 'tienda [master]' with a folder structure including 'Source Packages', 'Test Packages', and 'Other Sources'. The 'Other Sources' folder is expanded, showing 'src/main/resources' and a sub-folder '<default package>'. The 'application.properties' file is selected in the '<default package>' folder. The code editor shows the following content:

```
13  spring.thymeleaf.cache=false
14
15  #Conexión local a la BD Techshop en MySQL
16  #spring.datasource.url=jdbc:mysql://localhost:3306/techshop
17
18  #Conexión nube aiven a la BD Techshop en MySQL
19  spring.datasource.url=jdbc:mysql://techshop-techshop-40159.i.aivencloud.com:24382/techshop
20
```